Anyway, the following listing shows a simple implementation of a barrier.

Listing 8.12. A simple barrier class

```
class barrier
{
    unsigned const count;
    std::atomic<unsigned> spaces;
    std::atomic<unsigned> generation;
public:
    explicit barrier(unsigned count_):                    ❶
        count(count_),spaces(count),generation(0)
    {}
    void wait()
    {
        unsigned const my_generation=generation;          ❷
        if(!--spaces)                                     ❸
        {
            spaces=count;                                 ❹
            ++generation;                                 ❺
        }
        else
        {
            while(generation==my_generation)              ❻
                std::this_thread::yield();                ❼
        }
    }
};
```

With this implementation, you construct a `barrier` with the number of "seats" ❶, which is stored in the `count` variable. Initially, the number of `spaces` at the barrier is equal to this count. As each thread waits, the number of `spaces` is decremented ❸. When it reaches zero, the number of spaces is reset back to `count` ❹, and the `generation` is increased to signal to the other threads that they can continue ❺. If the number of free `spaces` does not reach zero, you have to wait. This implementation uses a simple spin lock ❻, checking the `generation` against the value you retrieved at the beginning of `wait()` ❷. Because the `generation` is only updated when all the threads have reached the barrier ❺, you `yield()` while waiting ❼ so the waiting thread doesn't hog the CPU in a busy wait.